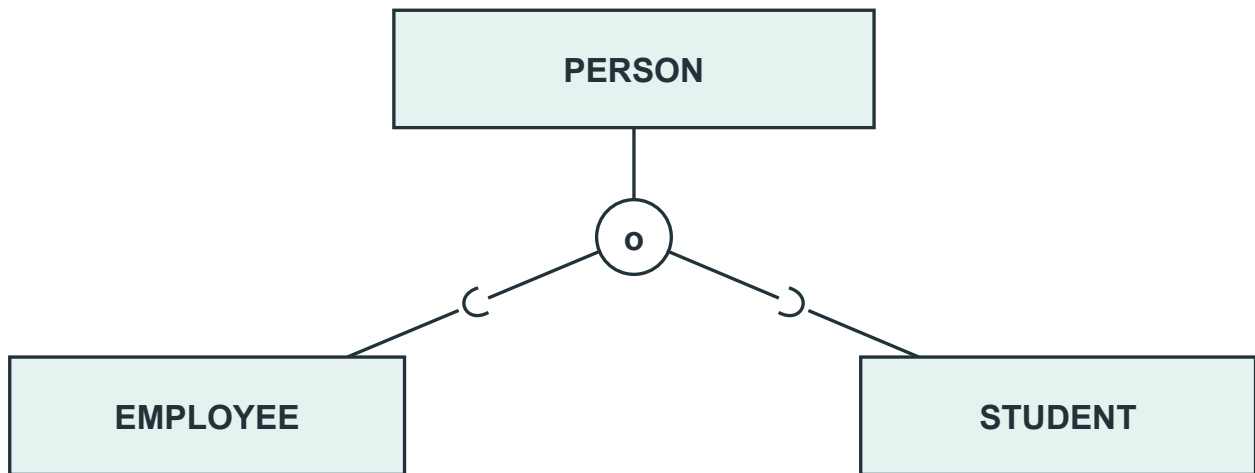


Chapter 4: Enhanced ER modeling

Keep the Ch3 entities and relationships. Add rules about different kinds of the same entity.



Example: a person may be an employee, a student, both, or neither.

Question	What we will draw
Is a student also a person?	Superclass, subclass, and inheritance
Can one person have two roles?	Disjointness and completeness constraints
Must a member belong to both groups?	Shared subclass versus category
Does this model match the description?	Data checks and repaired diagrams

Examples use invented data. Read their stated rules; do not assume every university uses them.

Draw from a description, repair a wrong diagram, and explain what the model allows.

Campus people: one person, several roles

A campus directory records people. Some work here, some study here, and some do both.

PERSON: PersonId	Name
P1	Alex
P2	Blair
P3	Casey
P4	Drew

Role	Current members	Extra information
EMPLOYEE	P1, P2	Salary
STUDENT	P2, P3	Major

Rules: both roles are allowed; neither role is also allowed. PersonId identifies the person across roles.

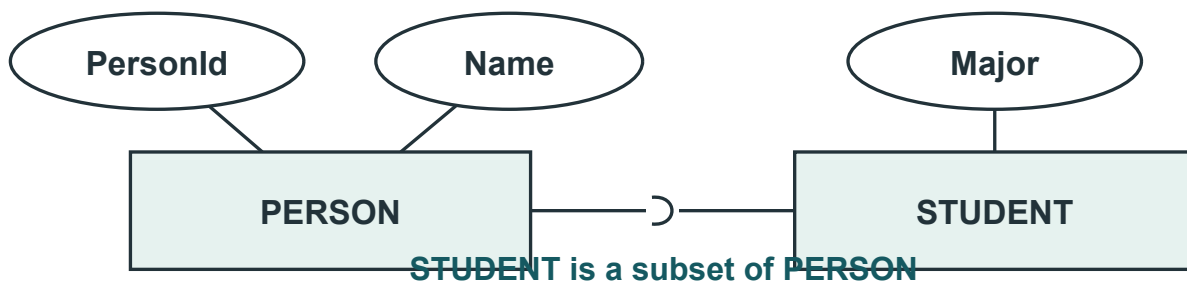
Predict: do we have four people or six?

Read the data: four people. P2 in two roles is still Blair, not two different people.

A subclass member is the same real-world entity as its superclass member.

A subclass inherits the person identity

STUDENT contains some of the people in PERSON. Major describes only students.



The curved subset mark opens toward PERSON. PersonId is the key; its underline is omitted here.

PERSON: PersonId	Name
P2	Blair
P3	Casey

STUDENT: PersonId	Major
P2	Information Management
P3	Accounting

Predict: does student P2 inherit Name? **Yes:** P2 is Blair. Do not invent a second identity.

PersonId is repeated in the display to match records. These tables show membership, not a required relational mapping.

Superclass: PERSON. Subclass: STUDENT. Every student is a person; not every person is a student.

Inherited relationships still apply

The campus library allows every registered PERSON to borrow books. Students inherit that possibility.



Here BORROWS means has borrowed at least once. Dates and repeat events are outside this fragment; participation is optional, attributes omitted.

Person	Student?	Borrowing history
P2	Yes	Copy C7
P4	No	None

Predict: can student P2 use BORROWS even without a second STUDENT-BORROWS diagram? **Yes.**

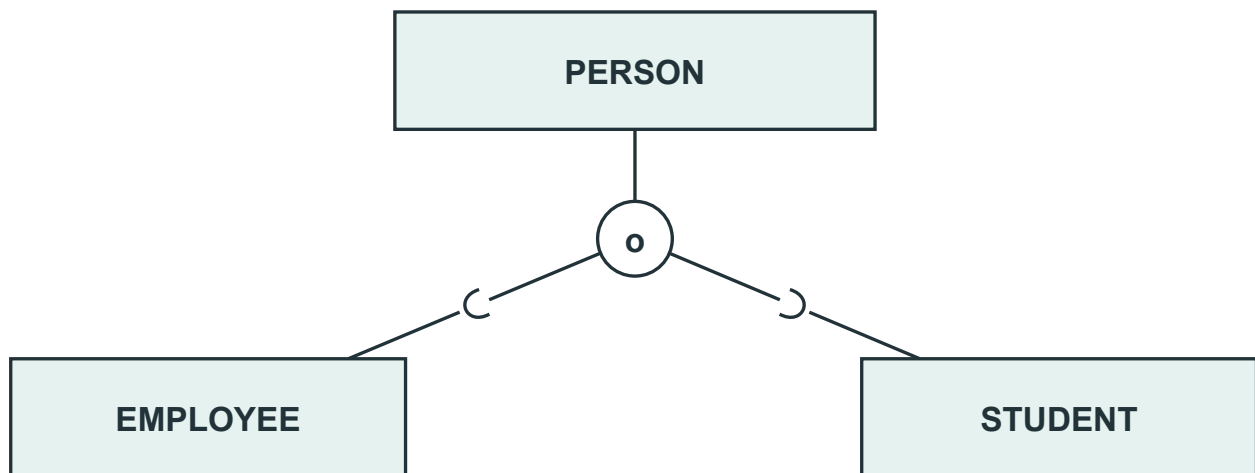
A subclass inherits the superclass attributes **and relationship participation.**

Inheriting a relationship does not create a borrowing record automatically. P4 is allowed to borrow but currently has none.

Inheritance gives the subclass the superclass properties, not a copy of another person or their borrowing records.

Specialization: start with the general type

Start with PERSON. Add EMPLOYEE for salary and STUDENT for major.



Read downward: general PERSON, then more specific roles. Attributes omitted.

Starting type	Subclass	What is specific?
PERSON	EMPLOYEE	Salary
PERSON	STUDENT	Major

Predict: should Name move into both subclasses? **No:** keep the common attribute on PERSON.

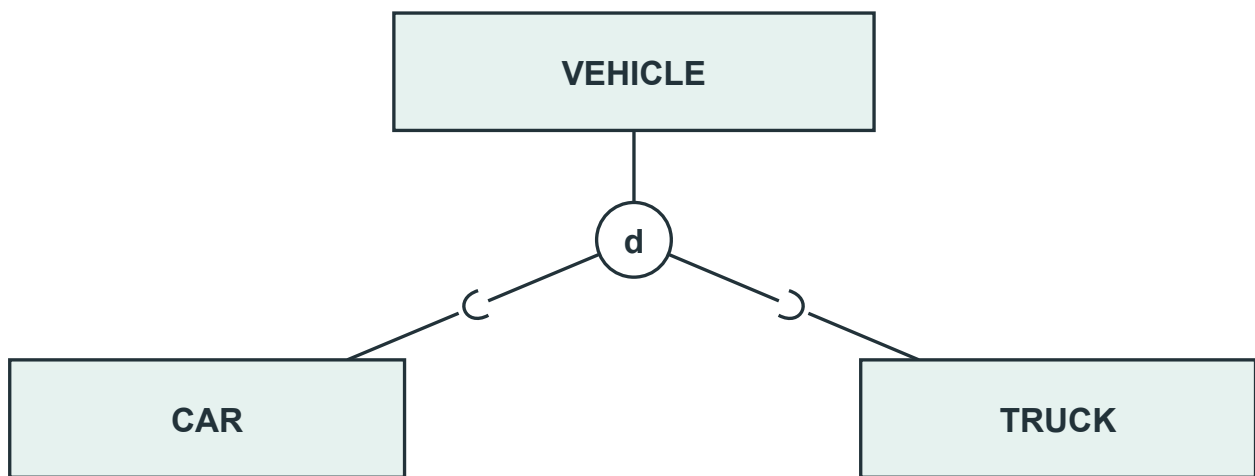
A subclass can also have its own relationships, such as a student enrolling in a section.

Specialization is a top-down design process. Add a subclass when its meaning and specific properties justify it.

Generalization: find the common type

Start with separate CAR and TRUCK descriptions. Both have a vehicle identity and license plate.

Original type	Common attributes	Specific attribute
CAR	VehicleId, Plate	Seats
TRUCK	VehicleId, Plate	LoadCapacity



Read upward: combine common properties in VEHICLE. This example allows other vehicle types; attributes omitted.

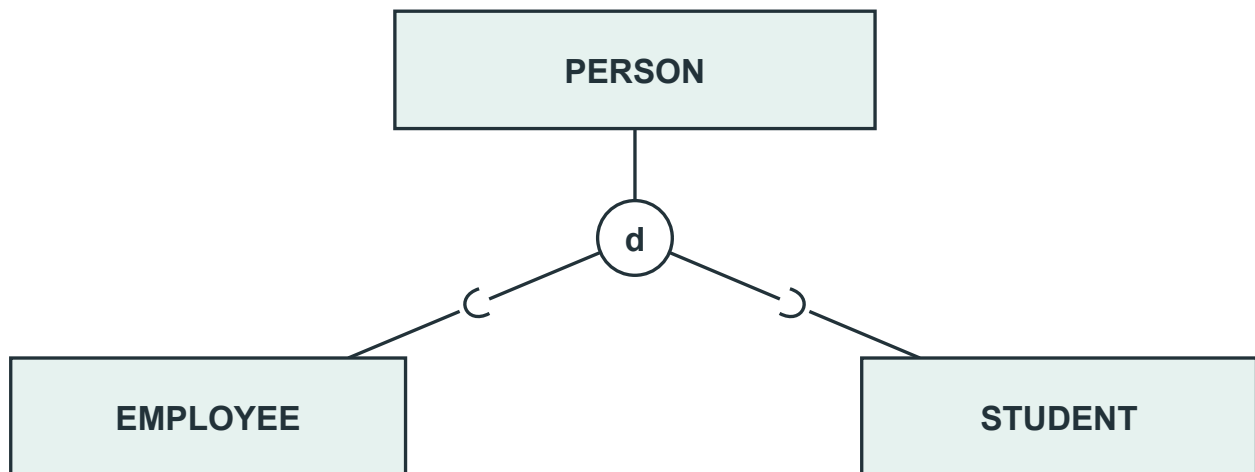
Rule: a vehicle is not both a car and a truck in this model.

Compare: specialization and generalization can lead to the same final diagram; the starting point differs.

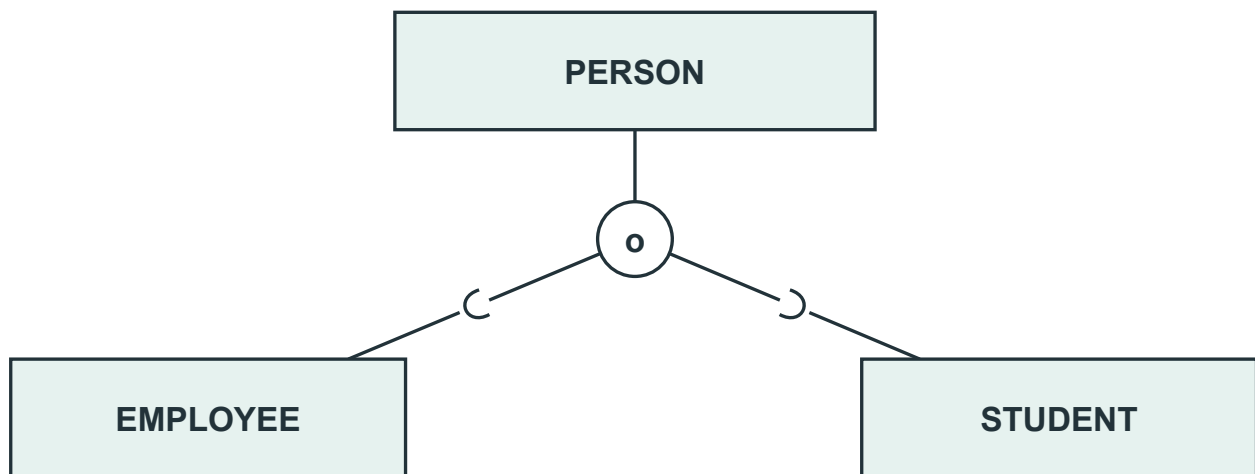
Generalization is bottom-up. The finished EER diagram does not reveal which design process was used.

Disjoint versus overlapping: can P2 be both?

Return to the campus directory: Blair is an employee and a student.



Wrong for these rules: d prohibits one person from joining both subclasses.



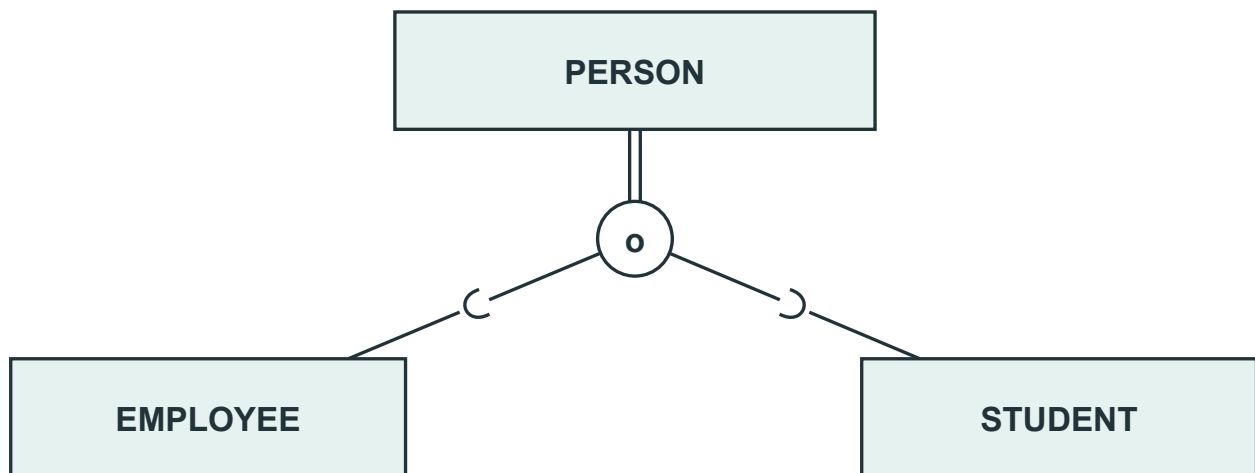
Repaired: o permits overlap. All other constraints stay the same.

Person	EMPLOYEE	STUDENT	Conflict with d?
P2	Yes	Yes	Yes

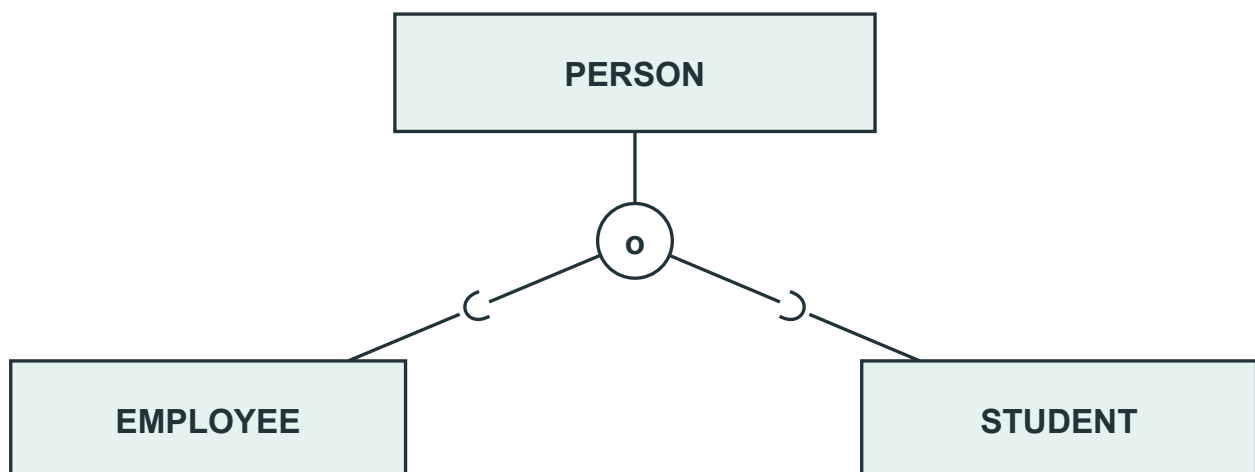
o permits overlap; it does not require every person, or any current person, to have both roles.

Partial versus total: can P4 have neither?

The directory also includes Drew, who is neither an employee nor a student.



Wrong for these rules: the double line requires every PERSON to have at least one listed role.



Repaired: the single line permits a PERSON outside both subclasses.

Person	EMPLOYEE	STUDENT	Conflict with total?
P4	No	No	Yes

Total means every superclass member is covered. Partial allows, but does not require, uncovered members.

Two independent questions, four combinations

Do not use d/o to answer whether a role is required. Do not use the line count to answer whether roles may overlap.

Constraint	Two roles allowed?	Neither allowed?
d + double line	No	No
d + single line	No	Yes
o + double line	Yes	No
o + single line	Yes	Yes

Campus fact	Required choice
P2 has both roles	o
P4 has neither role	Single line

Predict: can a total specialization use o? **Yes.** Everyone needs a role, but someone may have two.

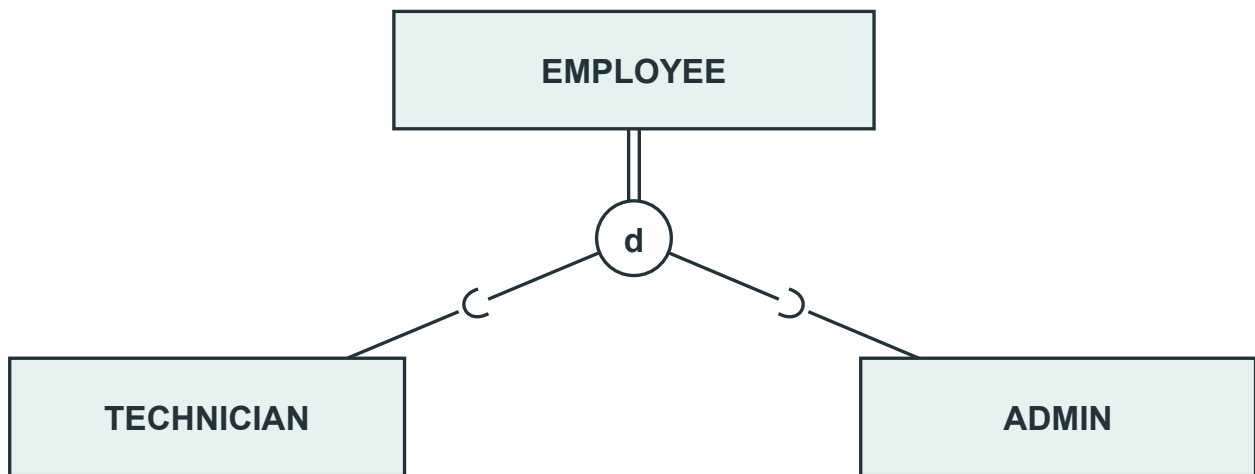
A small sample can expose a violation. It cannot prove that a different future membership is forbidden.

Choose constraints from the requirements, then test the displayed data against them.

A predicate can determine membership

An employee record has exactly one JobType. In this example, TECHNICIAN means JobType = Tech.

EMPLOYEE: PersonId	JobType	TECHNICIAN?
P1	Tech	Yes
P2	Admin	No
P5	Tech	Yes



Rule for this example: JobType is either Tech or Admin. The two subclasses use those exact values; attributes omitted.

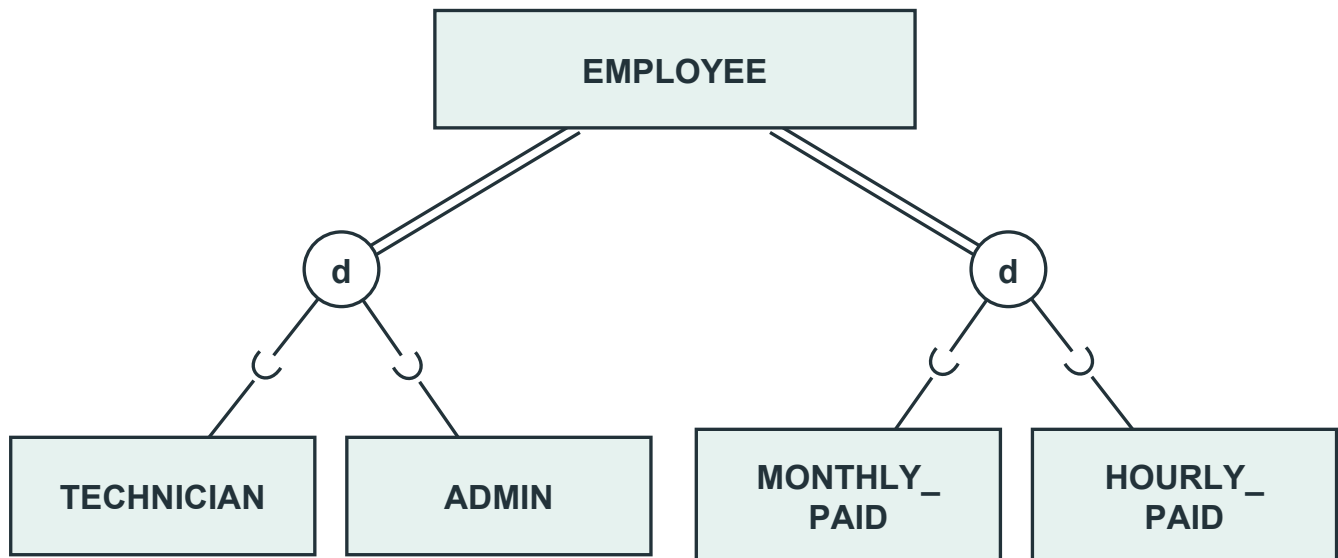
Predict: can P5 be omitted from TECHNICIAN? **No:** the predicate requires P5 to be included.

This is attribute-defined specialization: subclass predicates use the same single-valued attribute. Distinct values cannot both hold.

A predicate-defined subclass contains all and only the superclass members that satisfy its condition.

Two specializations answer different questions

An employee has one job type and one pay type. A technician may be paid monthly or hourly.



Each circle constrains its own subclass group. Both specializations are total and disjoint; attributes omitted.

Employee	Job group	Pay group
P1	TECHNICIAN	MONTHLY_PAID
P2	ADMIN	HOURLY_PAID

Wrong: putting all four subclasses under one d circle would reject P1 for having two memberships.

Repair: use separate specialization groups. P1 belongs to one subclass in each group.

Disjointness applies within its specialization, not automatically across every subclass of the superclass.

User-defined membership needs a decision

The campus chooses who joins a VOLUNTEER group. No stored attribute automatically determines membership.

PERSON: PersonId	Chosen for VOLUNTEER?
P1	Yes
P2	No
P3	Yes

Membership method	How is the subclass determined?
Predicate-defined	Evaluate an explicit condition for every person
User-defined	Record the authorized membership decision

Predict: can we infer that P2 is a volunteer from being a student? **No:** the rules do not say that.

Changing a predicate attribute may change subclass membership. For user-defined membership, record the separate decision.

Do not invent a condition from the current sample. Ask how membership is actually determined.

Membership changes must preserve the rules

Use the earlier total, disjoint EMPLOYEE split: TECHNICIAN or ADMIN.

Proposed change	Valid?	Why?
Add P6 to EMPLOYEE only	No	Total requires a subclass
Put P6 in both subclasses	No	d prohibits overlap
JobType changes from Tech to Admin	Yes, with role update	The predicate changes membership
Remove P1 from EMPLOYEE; keep it in TECHNICIAN	No	A subclass member must be in the superclass

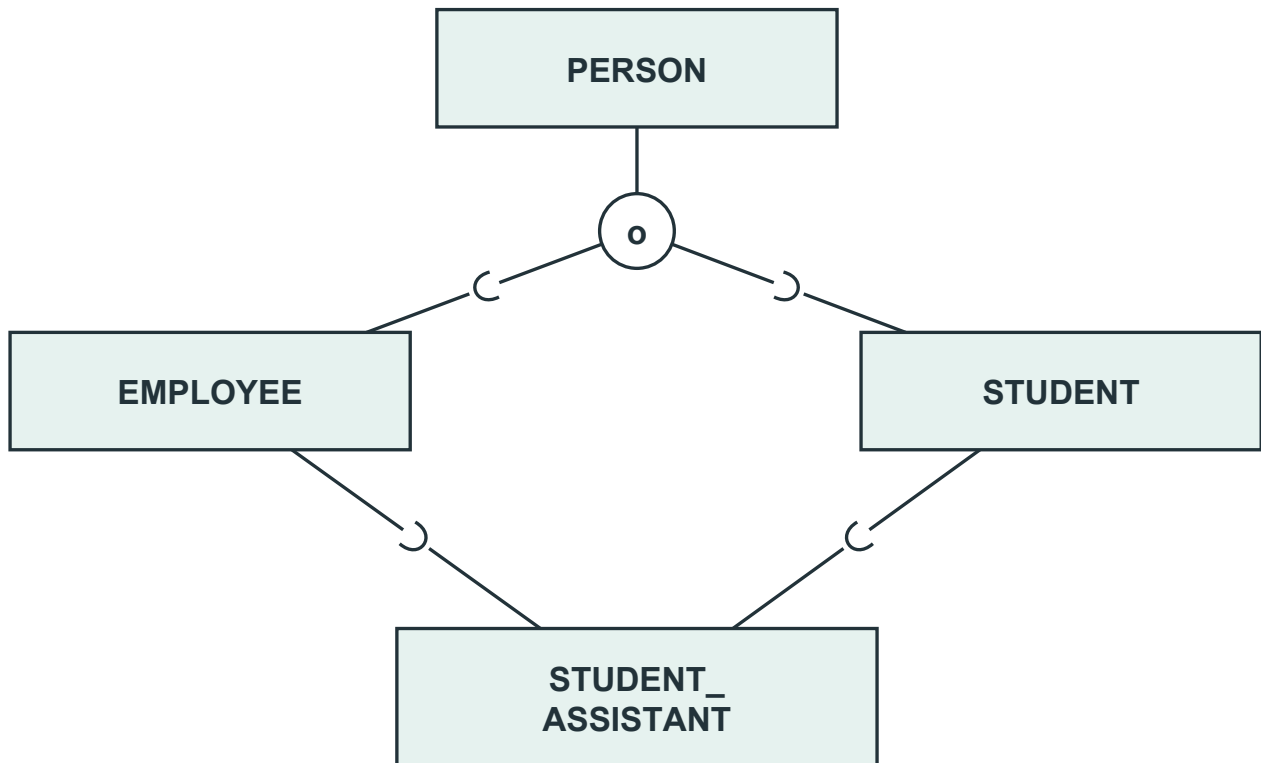
Discuss: repair the first change. Add P6 with a valid JobType and the matching subclass membership.

These are conceptual consistency rules, not instructions to run SQL DELETE or configure a database cascade.

A valid state must satisfy subclass membership, disjointness, completeness, and any predicates together.

A shared subclass requires both parents

The campus appoints some students as paid student assistants. Each assistant must be both a student and an employee.



STUDENT_ASSISTANT has two superclass links. Attributes omitted; PERSON roles are overlapping and partial.

Person	EMPLOYEE	STUDENT	Appointed assistant
P1	Yes	No	No
P2	Yes	Yes	Yes
P3	No	Yes	No

Predict: can P3 join STUDENT_ASSISTANT without joining EMPLOYEE? **No.**

A shared subclass is a subset of every superclass. It requires both memberships, not either membership.

Being in both groups is not an appointment

Add Evan, another employee who also studies here. Evan has not been appointed as a student assistant.

Person	EMPLOYEE	STUDENT	STUDENT_ASSIS TANT
P2: Blair	Yes	Yes	Yes
P5: Evan	Yes	Yes	No

Predict: does the shared-subclass diagram force P5 into STUDENT_ASSISTANT? **No.**

Statement	Correct?
Every assistant is both an employee and a student.	Yes
Every employee who studies is an assistant.	Not without an extra rule

The assistant inherits employee and student properties, and PERSON properties through both paths. P2 still has one PersonId and one Name.

Shared-subclass membership is contained in the intersection; it need not equal the whole intersection.

Hierarchy and lattice describe the links

A hierarchy gives each subclass one direct superclass. A lattice permits a subclass with multiple direct superclasses.

Model fragment	Direct parents	Structure
PERSON -> STUDENT -> GRAD_STUDENT	One at each step	Hierarchy
EMPLOYEE + STUDENT -> STUDENT_ASSISTANT	Two for the assistant	Lattice

Refine the design: move a common Name to PERSON; leave Salary on EMPLOYEE and Major on STUDENT.

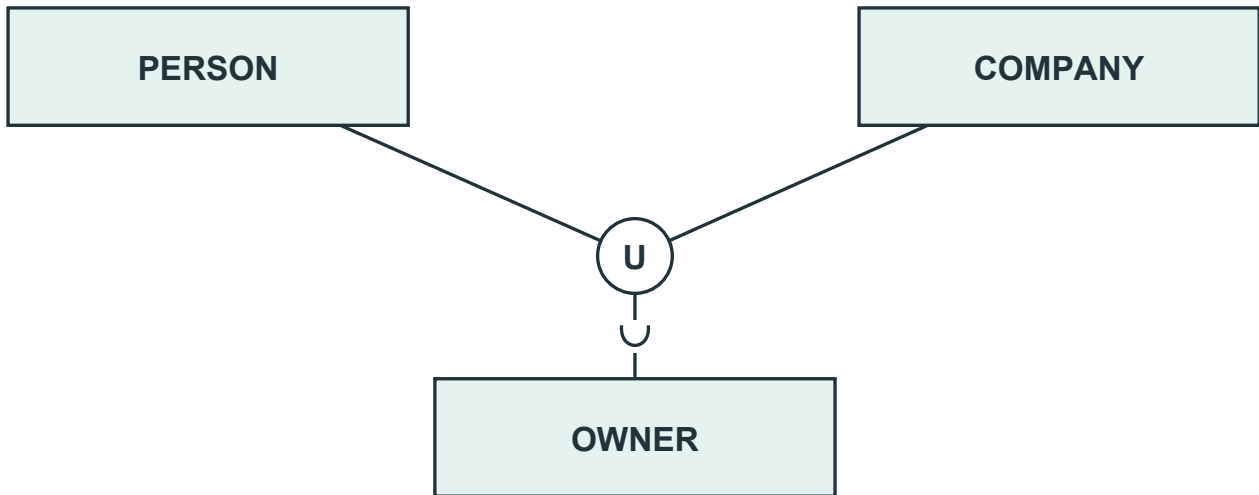
Predict: does adding a second superclass create a second person? **No:** it adds another required membership.

Use shared subclasses only when the requirements demand every parent membership. A complicated diagram is not automatically a more accurate design.

Inheritance is transitive: a graduate student inherits the properties of STUDENT and PERSON.

A category accepts members from a union

A vehicle registry records owners. An owner may be a person or a company; not every person or company owns a vehicle.



U is the union circle. OWNER is a partial category; attributes omitted.

Source entity	Source identity	Selected as OWNER?
PERSON	P1	Yes
PERSON	P2	No
COMPANY	C1	Yes

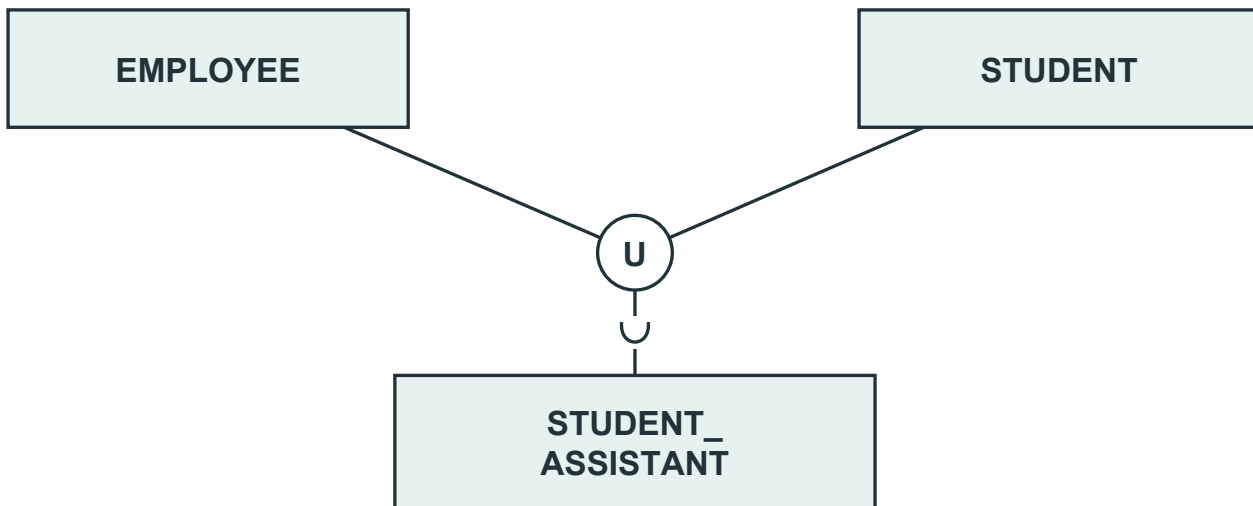
Predict: must company C1 also be a PERSON? **No:** either source type is enough.

An OWNER cannot come from outside the union. Its properties come from the source entity to which it belongs.

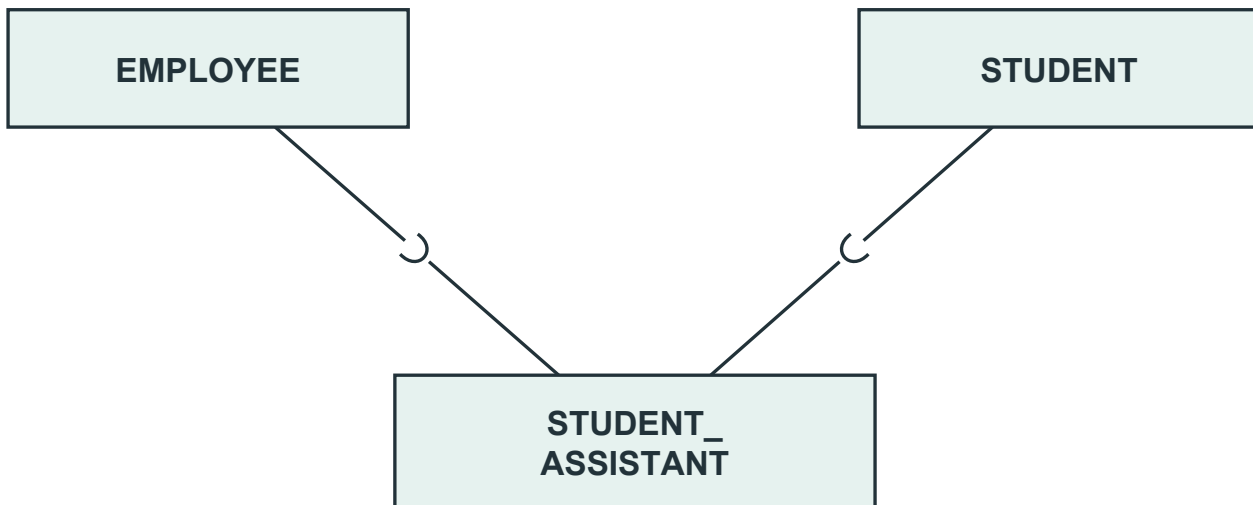
A category is a subset of a union. A shared subclass is a subset of an intersection.

Wrong union: an assistant needs both roles

Keep the assistant requirement unchanged: every assistant is both an employee and a student.



Wrong: a category allows a selected member from either source group.



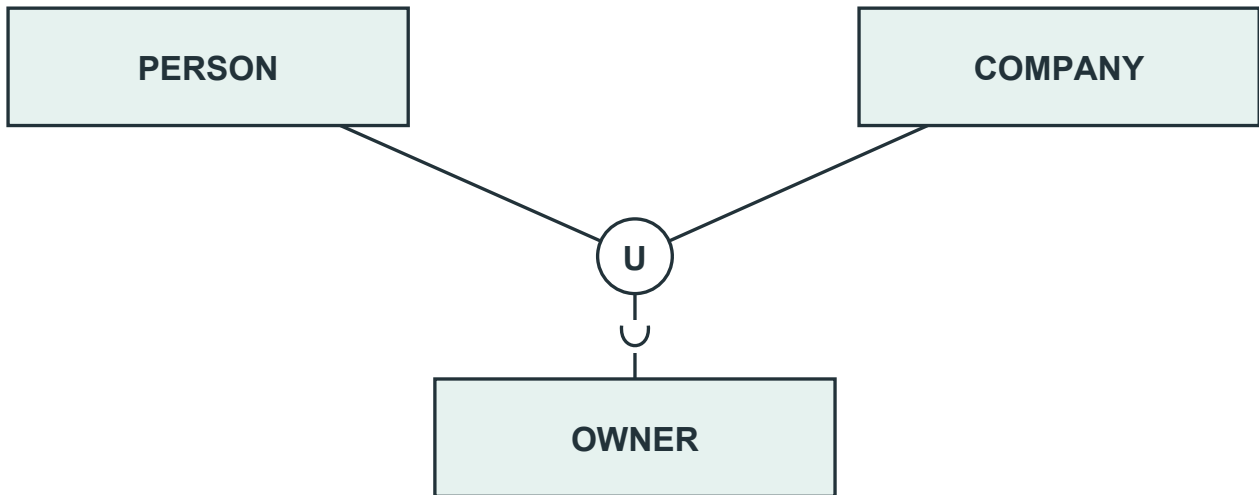
Repaired: two superclass links require both memberships.

Proposed assistant	EMPLOYEE?	STUDENT?	Meets requirement?
P1	Yes	No	No

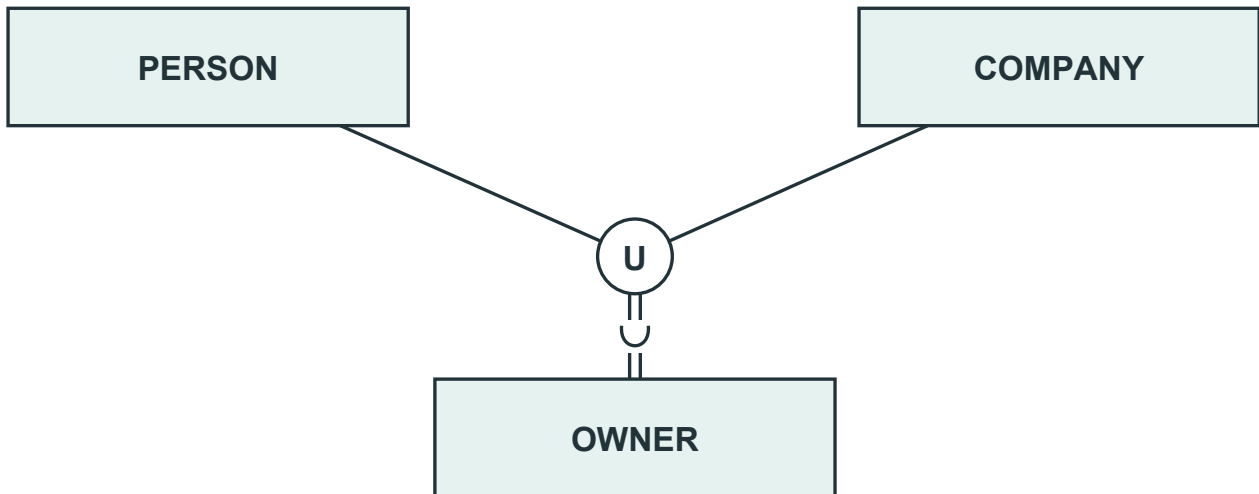
Ask whether the description says **BOTH** or **EITHER**. A union circle is not a shortcut for multiple inheritance.

Total category: all source members are included

Compare two possible vehicle registries. Only the category completeness rule changes.



A: partial category. Select the people and companies that are owners.



B: total category. Every source PERSON and COMPANY must belong to OWNER.

Counterexample for B: PERSON P2 owns no vehicle and is not an OWNER. A allows P2; B does not.

The double line is between OWNER and the union circle. Do not confuse this with a specialization double line.

Category identity comes from its source

The registry uses PersonId for people and CompanyNo for companies. The identifier formats can differ.

OWNER source	Source key	Inherited information
PERSON	P1	Person name: Alex
COMPANY	C1	Company name: North Ltd.

Predict: must OWNER have every PERSON attribute and every COMPANY attribute? **No.** A company owner inherits company properties.

The union itself does not say that the source classes are disjoint. If the source types overlap, a member can belong to more than one.

Unlike a general superclass, a category cannot contain an entity from outside its source union. Relational key mapping comes in Ch9.

Do not invent one universal source key or an exclusive-or rule unless the requirements supply it.

University case: define the requirements first

A small teaching-and-research office records people, sections, and grants. The next two diagrams form its complete model for this stated scope.

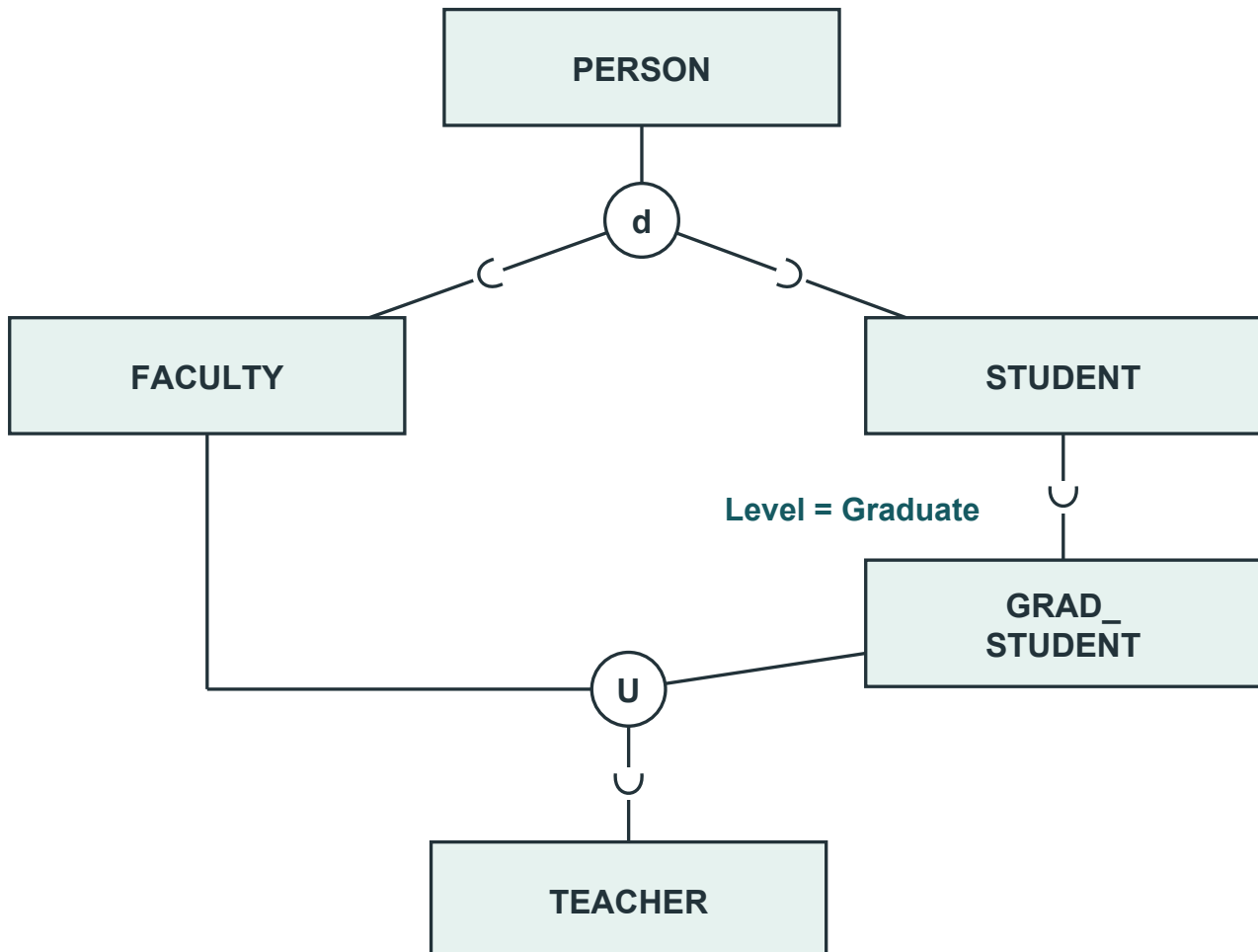
Object or role	Rule for this case
PERSON	May be FACULTY, STUDENT, or neither; never both
GRAD_STUDENT	A STUDENT with Level = Graduate
TEACHER	All faculty plus selected graduate students
SECTION	Exactly one teacher; a teacher may teach many
GRANT	Exactly one faculty leader; may support many graduates
GRAD_STUDENT	May receive support from many grants

This is a smaller, original adaptation of the textbook university example. Departments, colleges, courses, and advising are outside this case.

These are different requirements from our overlapping campus directory. Do not carry its o constraint into this case.

University case: the complete role structure

TEACHER is a category drawn from FACULTY and GRAD_STUDENT.
STUDENT and FACULTY are disjoint in this case.



Keys and other attributes appear in the following data tables. U alone does not encode the extra rule that all faculty are teachers.

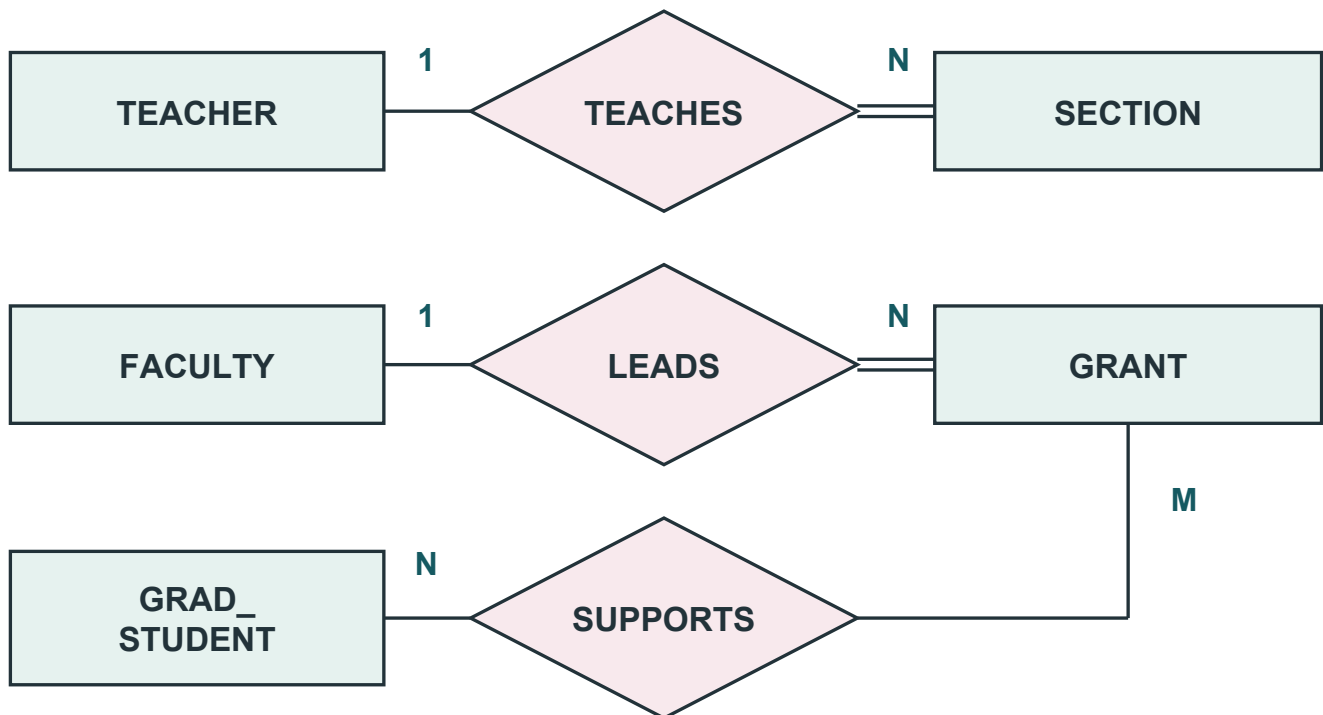
Additional rule: every FACULTY member belongs to TEACHER. Graduate students join TEACHER only when selected.

Predict: can an undergraduate be a teacher? **No:** an undergraduate is outside both source groups.

Keep rules that the symbols cannot express in plain language beside the diagram.

University case: the complete relationship structure

Use exactly the same TEACHER, FACULTY, and GRAD_STUDENT types from the role diagram.



These are the three relationships in this case. SUPPORTS links grants to graduate students. Attributes omitted.

Relationship	Read one participant at a time
TEACHES	One teacher may teach many sections; each section has one teacher
LEADS	One faculty member may lead many grants; each grant has one leader
SUPPORTS	A grant may support many graduates; a graduate may have many grants

The two university diagrams share types; they are two views of one model, not two separate databases.

University case: person and role records

PersonId identifies people throughout this case. Role rows show which of those same people belong to each type.

PERSON: PersonId	Name
U1	Lee
U2	Min
U3	Sam
U4	Kai
U5	Robin

Type	Members and local attributes
FACULTY	U1: Rank = Professor
STUDENT	U2: Level = Graduate; U3: Level = Undergraduate; U4: Level = Graduate
GRAD_STUDENT	U2: Program = MS; U4: Program = PhD
TEACHER	U1, U2

Check: U4 is a graduate student but not a teacher. U5 has neither faculty nor student membership.

The data satisfy partial, disjoint PERSON roles and a partial TEACHER category.

University case: section and grant records

SecId and GrantId uniquely identify sections and grants in this case.

SECTION: SecId	Term
S10	Fall 2026
S20	Fall 2026

GRANT: GrantId	Title
G1	Library study
G2	Learning study

Relationship	Current pairs
TEACHES (Teacher, Section)	(U1, S10), (U2, S20)
LEADS (Faculty, Grant)	(U1, G1), (U1, G2)
SUPPORTS (Grant, Graduate)	(G1, U2), (G2, U2), (G1, U4)

Predict: does U4 receiving grant support force U4 into TEACHER? **No:** our selection rule does not say that.

Use actual relationship pairs to test the role diagram and the cardinalities together.

Repair TEACHES: faculty alone is too narrow

The requirements allow selected graduate students to teach. U2 teaches section S20.



Wrong for this case: only FACULTY can occupy the teacher role.



Repaired: TEACHER includes faculty and selected graduate students.

Pair to represent	Faculty-only model	Category model
(U2, S20)	Rejected: U2 is not faculty	Allowed: U2 is a teacher

Do not repair this by placing U2 in FACULTY: that would violate the stated disjoint roles.

Repair the model against the requirements, not by changing a person into the wrong type.

CURRENT_SECTION needs the full condition

Extend the same university model with a predicate-defined subclass. Current means Fall 2026, not every Fall.

SECTION: SecId	Quarter	Year	CURRENT_SECTION?
S10	Fall	2026	Yes
S20	Fall	2026	Yes
S30	Fall	2025	No
S40	Spring	2026	No

Predicate	Problem or result
Quarter = Fall	Wrong: also selects S30
Quarter = Fall AND Year = 2026	Correct: selects S10 and S20

CURRENT_SECTION inherits SECTION identity and TEACHES participation. It does not create a second section.

The textbook figure uses numeric Class values for graduates, but its figure and prose disagree. This example explicitly uses Level = Graduate.

Write every condition needed to determine membership; do not let an unexplained code decide the rule.

Choose a subclass only when it helps the model

A library records a member preference. Compare this with a student role that has its own attributes and relationships.

Requirement	Reasonable representation	Reason
Preferred contact is Email or Phone	Attribute: ContactPreference	No separate role behavior is specified
Students have Major and ENROLLS_IN	Subclass: STUDENT	Specific attributes and relationships
One employee has a different salary	Attribute value: Salary	A different value alone need not create a type

Discuss: should every possible salary become a subclass? **No:** that would add types without a useful requirement.

Several designs may preserve the requirements. Compare what they express; do not choose only by diagram size.

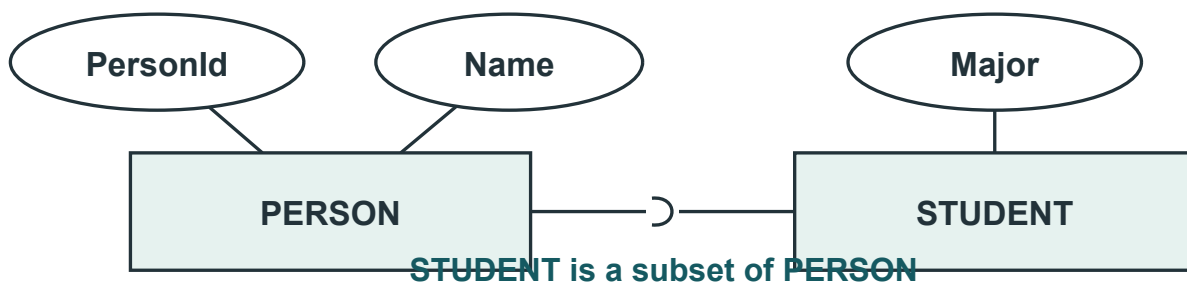
Put common properties in the superclass; introduce subclasses for meaningful distinctions that the application needs.

Subclass is not a weak entity

STUDENT is a PERSON. A Ch3 SECTION belongs to a COURSE but is not itself a course.



Wrong explanation of inheritance: an ordinary HAS relationship does not assert that STUDENT is the same entity as PERSON.



Repaired: a superclass/subclass link states that each STUDENT is a PERSON.

Type	Identity meaning
STUDENT P2	The same person P2
Weak SECTION DB101 / 1	A separate section identified using its course and SectionNo

Inheritance means IS A. Weak-entity identification means NEEDS ITS OWNER TO BE IDENTIFIED.

Read the formal definitions through the data

Use the campus directory: $P = \{P1, P2, P3, P4\}$, $E = \{P1, P2\}$, $S = \{P2, P3\}$, $A = \{P2\}$.

Notation	Meaning	Example result
E subset of P	Every employee is a person	P1 and P2 occur in P
E union S	In E, S, or both	$\{P1, P2, P3\}$
E intersection S	In both E and S	$\{P2\}$
A subset of (E intersection S)	Every assistant has both roles	P2 qualifies

Disjoint: the subclass intersection is empty. **Total:** the subclass union equals the superclass.

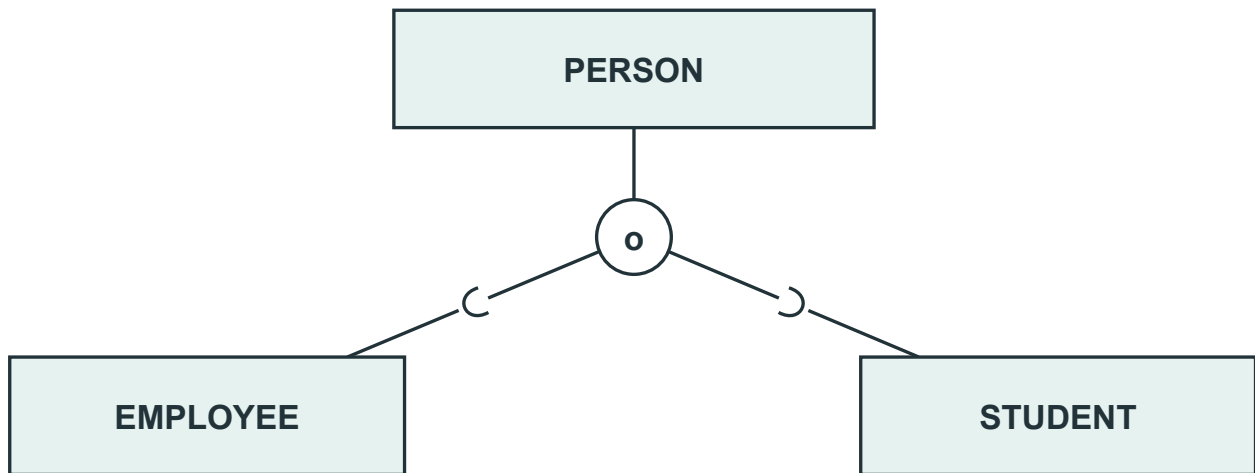
Check this data: the intersection contains P2, so d fails. The union misses P4, so total fails.

For a category T drawn from E and S: T is a subset of their union. Total category means equality.

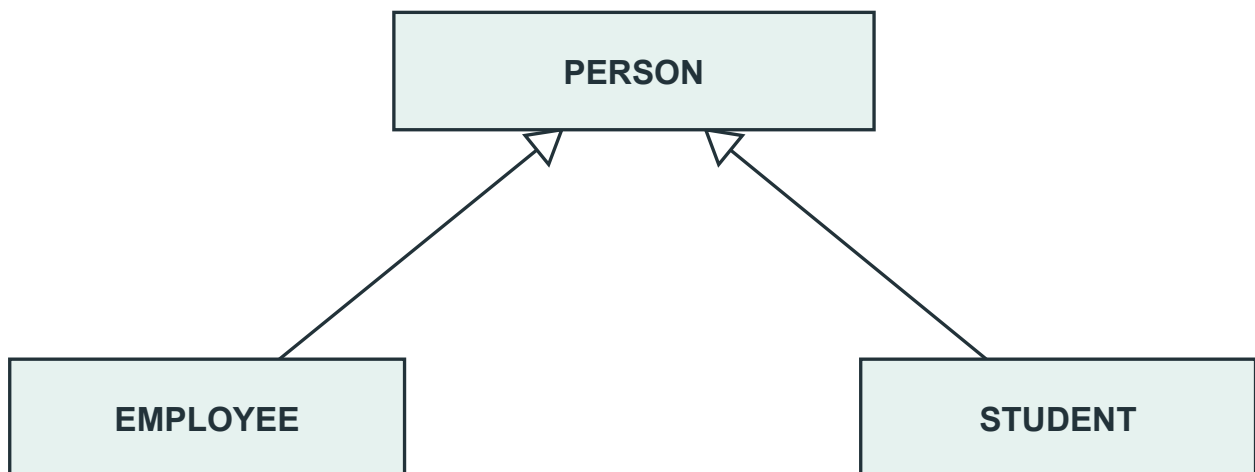
The set expressions restate membership rules; they are not additional diagram rules.

UML: the same roles, different notation

Translate the overlapping, partial campus directory into a UML class diagram without changing its rules.



EER: o and a single superclass line mean overlapping and partial.



UML: hollow triangles point toward the superclass. Generalization set: {incomplete, overlapping}.

The constraint belongs to the pair of generalizations. P2 may have both roles; P4 may have neither.

For UML 2.5.1, use hollow triangles and explicit constraint text. A filled triangle is not the overlapping symbol.

UML constraints must preserve the EER rules

Translate the rules, not just the shapes. In this table, complete corresponds to total and incomplete to partial.

EER specialization	UML generalization-set constraint
d + double line	{complete, disjoint}
d + single line	{incomplete, disjoint}
o + double line	{complete, overlapping}
o + single line	{incomplete, overlapping}

Wrong: marking our campus roles {complete, disjoint} excludes both P2 and P4.

Repair: use {incomplete, overlapping}; the membership rules now match the EER model.

Textbook Figure 4.10 uses a triangle-fill convention that differs from UML 2.5.1. Here the explicit constraint text determines the meaning.

Do not import an EER circle or the textbook triangle-fill convention into a standard UML diagram.

UML classes: three short distinctions

A UML class can show attributes and operations. Database conceptual design does not require us to implement those operations here.

Kind	Simple example	Meaning
Abstract class	SHAPE with CIRCLE and RECTANGLE	No direct SHAPE instances
Concrete class	CIRCLE	Can have direct instances
Template class	A collection parameterized by element type	A parameterized definition, not one object

Predict: does an abstract superclass mean every subclass has an instance today? **No.** It constrains direct instantiation, not current population.

Do not confuse a class, its current objects, and a program variable that refers to one object.

Use UML vocabulary to read the model. No application implementation is needed for this chapter.

Classification: type versus instance

Return to the library. MEMBER is a type; member M1 is one entity of that type.

Description	Level	Example
MEMBER	Class or entity type	Defines MemberId and Name
M1, Alex	Instance	One member with those values
A catalog of entity types	Metaclass example	MEMBER itself is one recorded type

Predict: is STUDENT a single student? **No:** it is a subclass describing a group of possible students.

Classification groups entities by type. Instantiation creates or identifies an instance of a type. Generalization groups types under a more general type.

Keep the type definition separate from the people or objects that currently belong to it.

Identification across roles and records

A library and a campus office must agree which real-world person a record represents.

MemberId	Name	Campus PersonId
M1	Alex Lee	P7
M2	Alex Lee	P8

Proposed match	What the data support
Match by Name only	Wrong: two people share the name
Use the recorded identity link	M1 refers to P7; M2 refers to P8

This is why the role examples preserve one PersonId across subclasses. A name match alone does not establish shared identity.

Class names also need clear identification: MEMBER and STUDENT are different type definitions even when their populations overlap.

An identity link must be established from reliable records, not inferred from similar labels.

Library case: model each borrowing event

The library keeps repeat loans of the same copy by the same member. It may issue several notices for one loan.

Requirement	Model decision
Each loan belongs to one member and one copy	LOAN links MEMBER and COPY
The same pair can borrow again later	LoanId identifies each event
Each notice concerns exactly one loan	NOTICE links to LOAN
Members and copies may have no loans yet	Their loan participation is partial

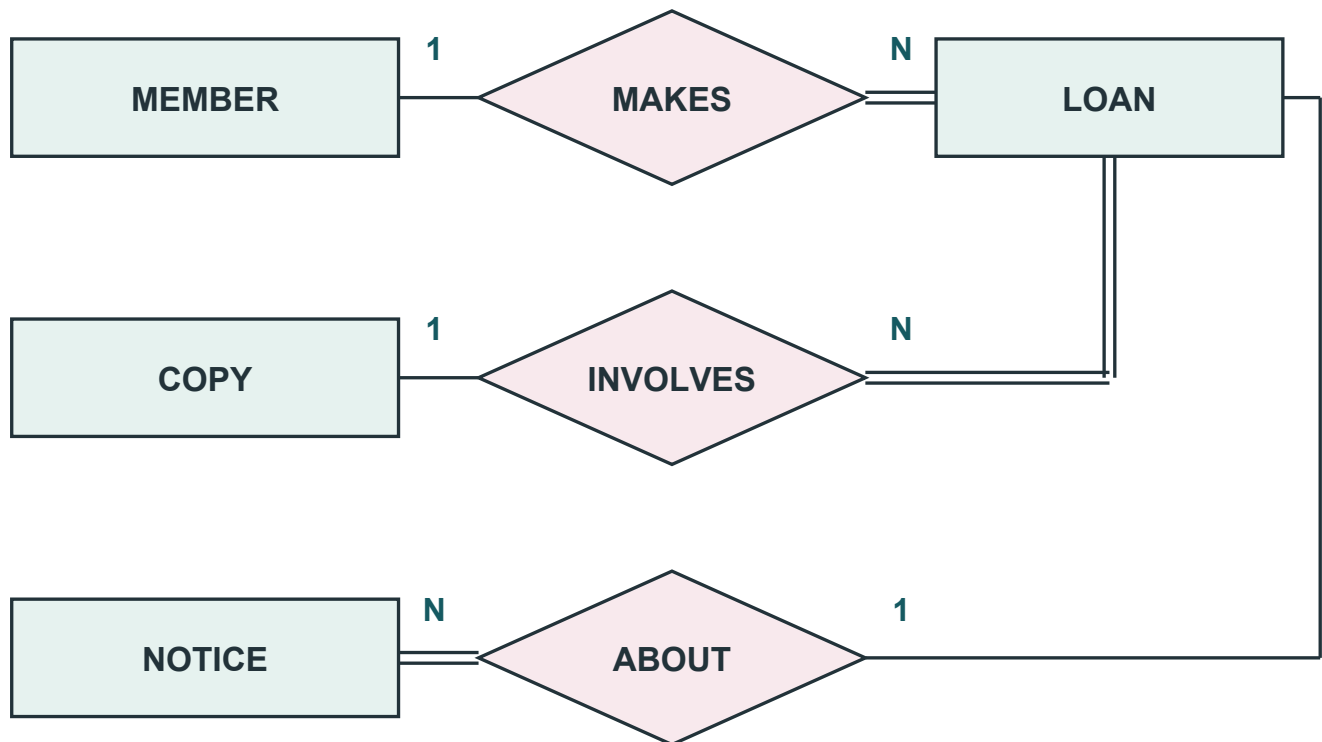
This extends the Ch3 library case: an event becomes an entity so that we can refer to that particular event.

Predict: can MemberId + CopyId distinguish two different loans by the same person? **No.**

When another relationship must refer to a specific event, give that event an explicit representation.

Library case: the complete ERD

LOAN is an event entity with its own LoanId. It is not a weak entity in this design.



Keys: MemberId, CopyId, LoanId, NoticeId. Attributes are omitted from the shapes and shown in the next tables.

Entity	Read its participation
LOAN	Exactly one MEMBER and one COPY
NOTICE	Exactly one LOAN
MEMBER / COPY	Zero or more loans
LOAN as notice target	Zero or more notices

Each relationship joins entity types. No relationship diamond is used as another relationship participant.

Library case: the records behind the diagram

All identifiers below are fictional. L1 and L2 involve the same member-copy pair but are different events.

MEMBER: MemberId		Name	
M1		Alex	
COPY: CopyId		Title	
C7		Database Design	
LOAN: LoanId	Member	Copy	Borrowed
L1	M1	C7	2026-03-01
L2	M1	C7	2026-04-01
NOTICE: NoticId	Loan		Sent
N1	L2		2026-04-20

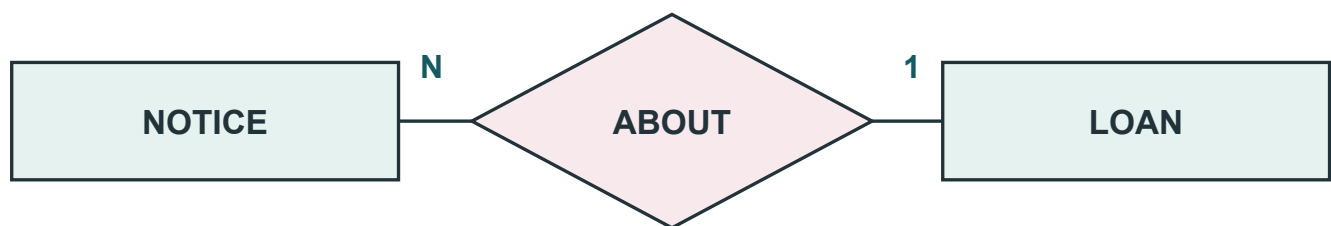
Check: N1 is about L2 only, not the earlier L1. Foreign-looking columns here display relationship facts; physical mapping comes later.

The event identity tells us exactly which occurrence another record refers to.

Repair the notice: point to the event

Keep the two loans L1 and L2. Compare what the notice stores.

Wrong NOTICE	Member	Copy	Which loan?
N1	M1	C7	L1 or L2: not determined



Repaired target: the loan event. The complete ERD shows total NOTICE participation; this fragment omits participation lines.

Repaired NOTICE	Loan	Which loan?
N1	L2	2026-04-01 borrowing

In ordinary ER/EER notation, do not connect ABOUT to a BORROWS diamond as though the diamond were an entity.

Represent the borrowing event as LOAN, then connect NOTICE to LOAN.

The repair preserves the event being referenced; it is more than changing a diagram symbol.

Aggregation has more than one use

The textbook discusses combining attributes, part-whole structures, and treating an association as a higher-level object.

Use	Concrete example	What to check
Combine properties	Street + City form Address	A composite attribute
Part-whole relationship	A car has an engine	State ownership and lifecycle rules
Association as an object	A borrowing event becomes LOAN	Other entities can refer to that event

Do not assume: deleting a whole always deletes every part. Ordinary part-of relationships do not automatically impose that rule.

UML composition is stronger: a part belongs to at most one composite at a time, and remaining object parts are deleted with it. State that meaning explicitly.

Explain which meaning of aggregation is intended; the word alone does not specify identity or deletion rules.

Ontology: shared meaning and stated rules

Suppose a library and a campus office share a vocabulary for Person and Student.

Shared statement	Example
Concept	Student is a kind of Person
Instance fact	Blair is a Student
Rule-based conclusion	Therefore Blair is a Person

Predict: can we conclude that Blair is an Employee? **No:** no fact or rule supports it.

Focus	What is being described?
Database conceptual model	Data and constraints needed by an application
Ontology	Explicit shared concepts, relationships, and meaning, potentially across applications

Knowledge representation may include inference rules and other knowledge beyond a database schema. The two areas overlap; they are not mutually exclusive.

Use the declared facts and rules. A shared vocabulary does not justify unstated conclusions.

Chapter summary: explain each modeling choice

Use the same three skills as Ch3. Keep each explanation tied to a requirement and a concrete entity.

Skill	Worked reminder
Draw from a description	Assistants are both students and employees: shared subclass
Repair an incorrect diagram	P2 has both roles: replace d with o for that requirement
Explain the diagram	TEACHER category permits a selected graduate student to teach

Keep: your annotated diagram and one data row that explains a constraint or exposes a wrong model.

Before the next chapter: distinguish a real-world entity, its type, and a table row. Ch5 introduces the relational model; Ch9 maps ER/EER designs to relations.

A diagram is useful when its symbols, stated rules, and example data tell the same story.

Reading and notation

Primary reading: Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, 7th edition, Chapter 4.

Topic	Textbook sections
Subclass and inheritance	4.1
Specialization and generalization	4.2
Constraints and shared subclasses	4.3
Categories	4.4
University example, choices, definitions	4.5
UML	4.6
Abstraction and ontology	4.7

The campus, registry, and library data and diagrams are original teaching examples, not reproduced textbook figures.

For UML notation, see OMG Unified Modeling Language 2.5.1, sections 9.2.4.2, 9.5.3, and 9.7.4.

The simplified university case states its own rules. Textbook Figure 4.7 permits overlapping PERSON roles; Figure 4.9 uses disjoint roles. Do not combine those assumptions.

Read the assumptions before comparing diagrams that use similar names.